Session 03 – Artificial Neural Networks

Hector J. Hortua, PhD

Presentacion basada en curso ML de Prof Ivan Olier

Learning outcomes

By the end of this session we should be able to:

- Describe a perceptron, its structure and learning algorithm.
- Understand the perceptron limitations and how they were overcome.
- Design MLP architectures depending on the problem to solve.
- Implement the perceptron and the multi-layer perceptron in Python.

Definition of (artificial) neural networks

- A neural network is a massively parallel distributed processor made up of simple processing units that has a natural propensity for storing experiential knowledge and making it available for use.

- It resembles the brain in two respects:
  1. Knowledge is acquired by the network from its environment through a learning process.
  2. Inter-neuron connection strengths, known as synaptic weights, are used to store the acquired knowledge.

From linear regression to neurons

Linear regression:

$$y = b + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m$$

$$v = b + \sum_{j=1}^{m} w_j x_j$$

$$y = v$$

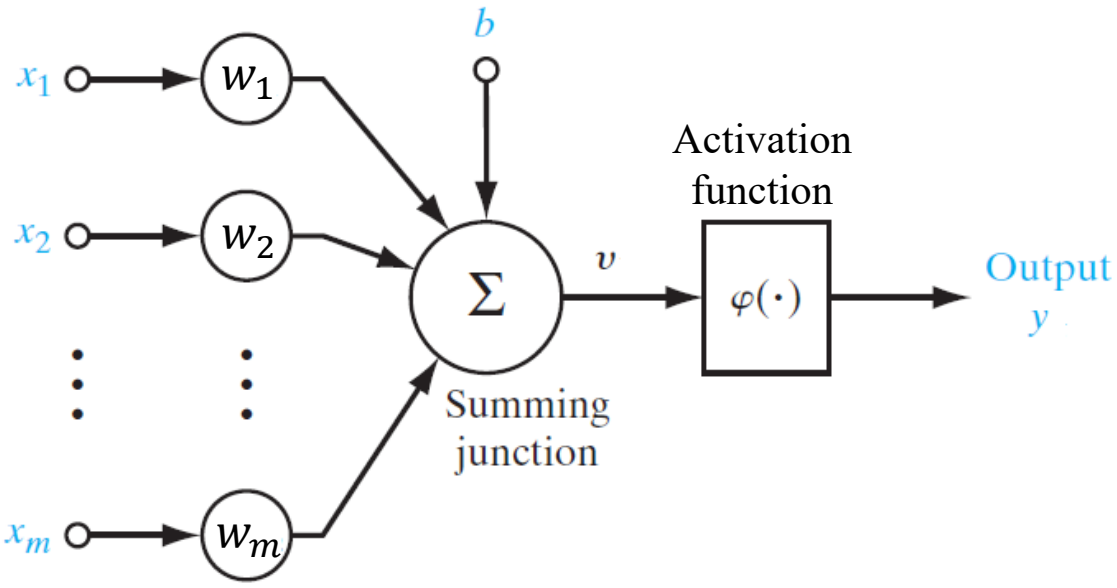
Logistic regression:

$$y = p(x) = \frac{e^{b + w_1 x_1 + \cdots w_m x_m}}{1 + e^{b + w_1 x_1 + \cdots w_m x_m}}$$

$$y = p(x) = \frac{1}{1 + e^{-(b + w_1 x_1 + \cdots w_m x_m)}}$$

$$y = \varphi(v) = \frac{1}{1 + e^{-v}}$$

Graphical representation of the models



This is a **neuron** – a simple processing that performs:

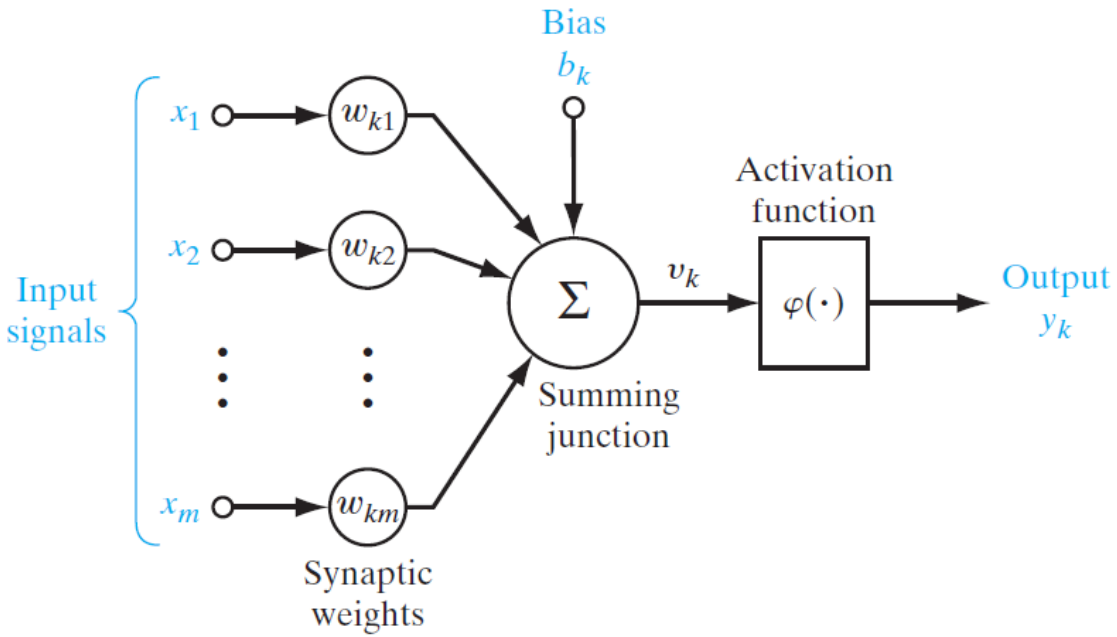
1. Linear combination (adder) of inputs + bias
2. (Non)-linear activation function (φ) to produce the output.

Models of neurons

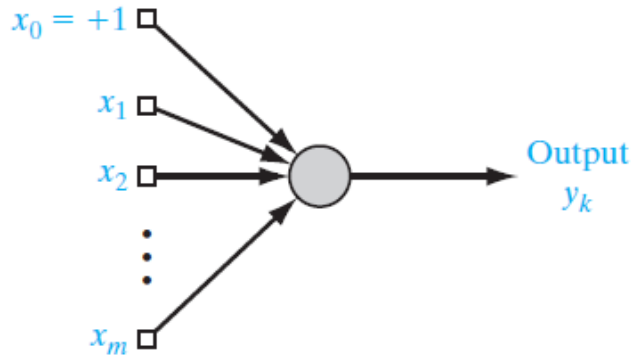
$$v = y = b + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m$$

$$v_k = b_k + \sum_{j=1}^{m} w_{kj} x_j$$

output signal of the adder



Architectural graph of a neuron



$$y_k = \varphi(v_k)$$

$$= \varphi\left(b_k + \sum_{j=1}^{m} w_{kj} x_j\right)$$

$$= \varphi\left(\mathbf{w}_k^T \mathbf{x}\right)$$

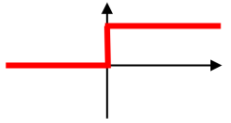
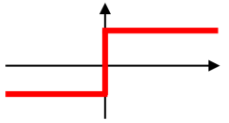
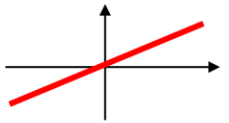
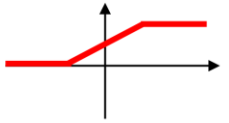
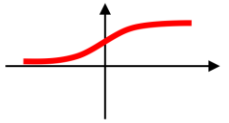
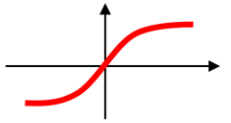
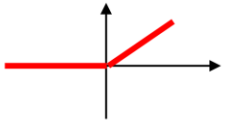
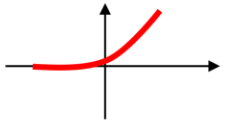
output signal of the neuron

In matrix form:

$$v_k = [b_k, w_{k1}, \ldots, w_{km}] \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_j \end{bmatrix} = \mathbf{w}_k^T \mathbf{x}$$

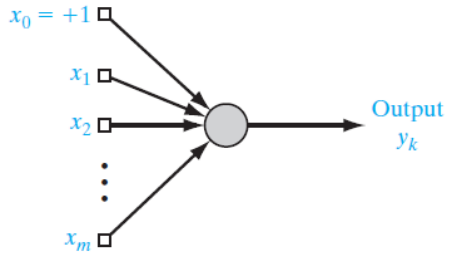
(dot product)

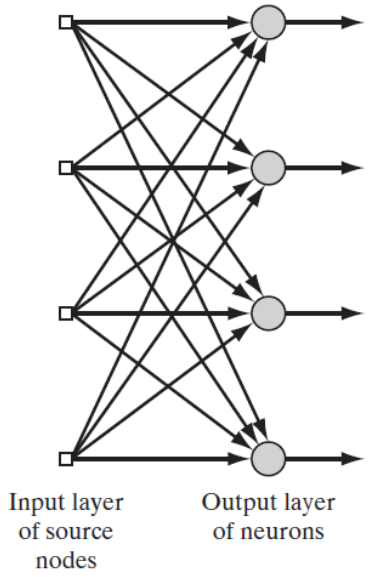
Summary of commonly used activation functions

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^{z})$	Multi-layer Neural Networks	

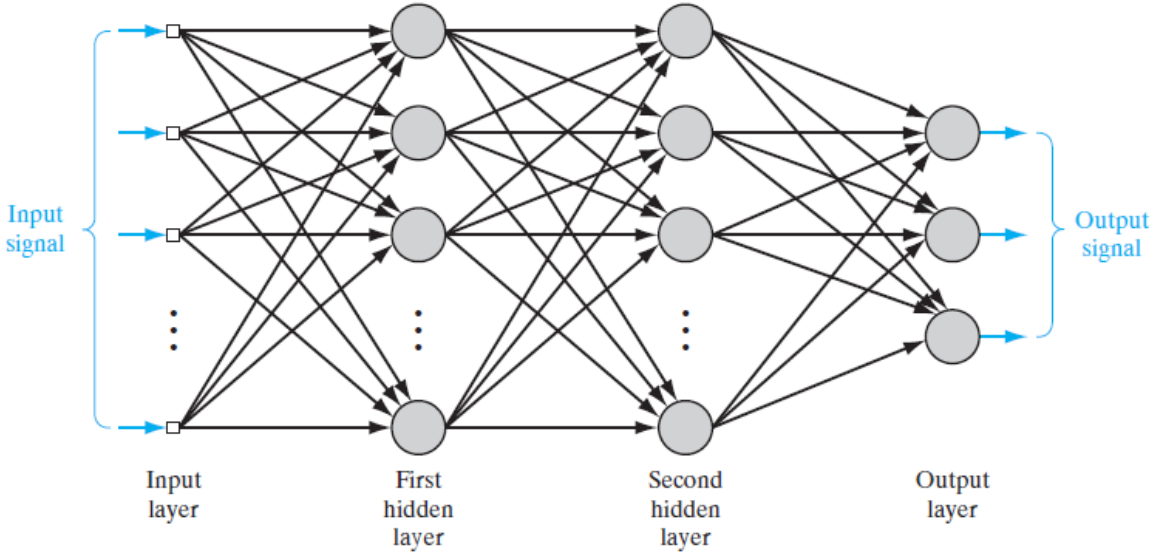
From neurons to neural networks

- A neural network consists of several (could be thousands) interlinked neurons.





Multivariate multi-output models can be easily constructed by “vertically” stacking neurons

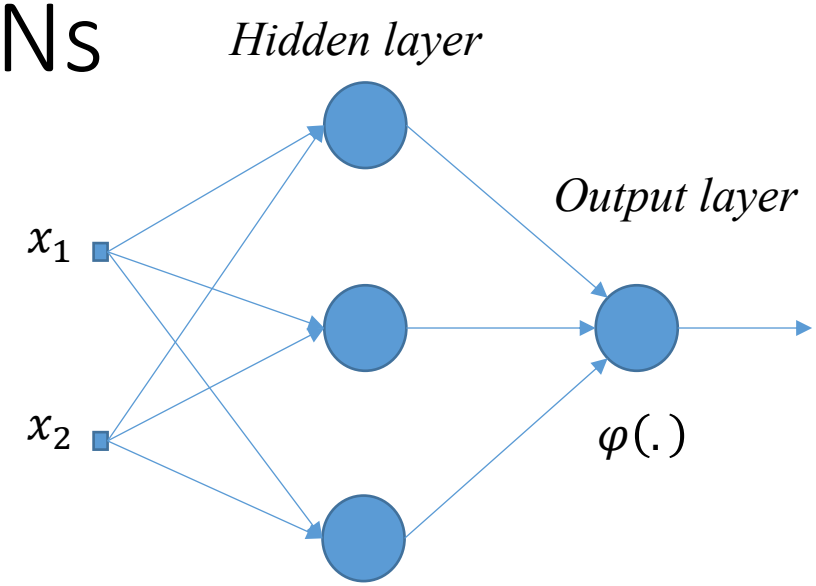


complex interaction are modelled by “horizontally” stacking layers of neurons

Benefits of neural networks

- inherently nonlinear
- the network learns from the examples by constructing an input–output mapping for the problem at hand.
- It has the capability to adapt their synaptic weights to changes in the surrounding environment.
- They can be designed to provide information not only about which particular pattern to select, but also about the confidence in the decision made.
- This may be used to reject ambiguous patterns, and thereby improve classification performance.
- Knowledge is represented by the very structure and activation state of a neural net.
- Every neuron in the network is potentially affected by the global activity of all other neurons in the network.
- Consequently, contextual information is dealt with naturally by a neural network.
- Neural networks can be implemented to **solve** regression, classification, clustering, blind source separation, novelty detection, amongst other kind of **data mining** problems.

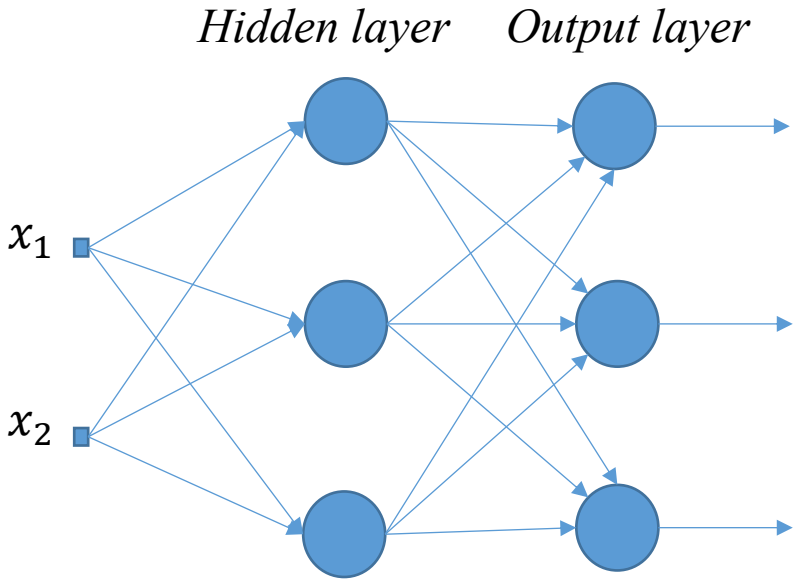
Regression and classification with NNs



- $\varphi(.)$ is linear for regression tasks (e.g. $\varphi(v) = v$)
- $\varphi(.)$ is either sigmoid, tanh, or equivalent, for a two-class classification task.

- A multi-output regression task is just an **extension** of the same architecture.
- It is also called "multi-task learning".
- This network can also be used for multi-class classification tasks.
- The *softmax* activation function is typically used. It is seen as a **generalised** sigmoid, so the outputs are still probabilities:

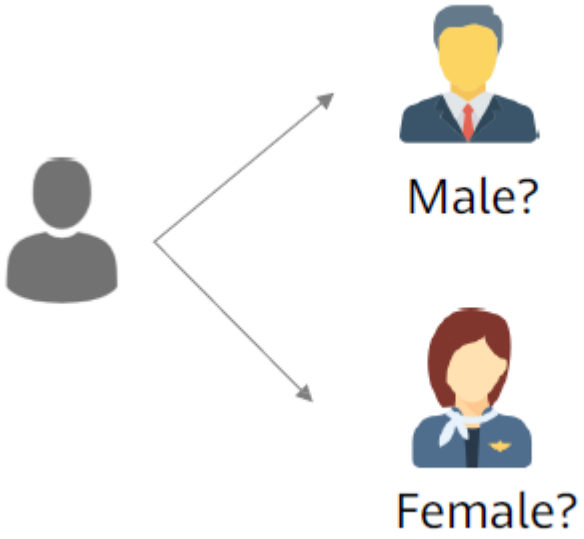
$$y_i = \frac{e^{v_i}}{\sum_{i=1}^{c} e^{v_i}}$$



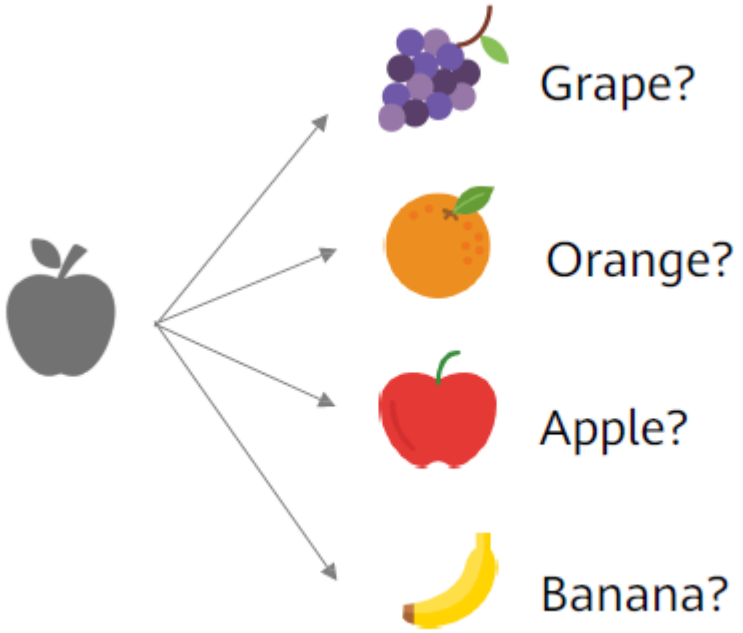
Logistic Regression Extension - Softmax Function (1)

- Logistic regression applies only to binary classification problems. For multi-class classification problems, use the Softmax function.

Binary classification problem



Multi-class classification problem



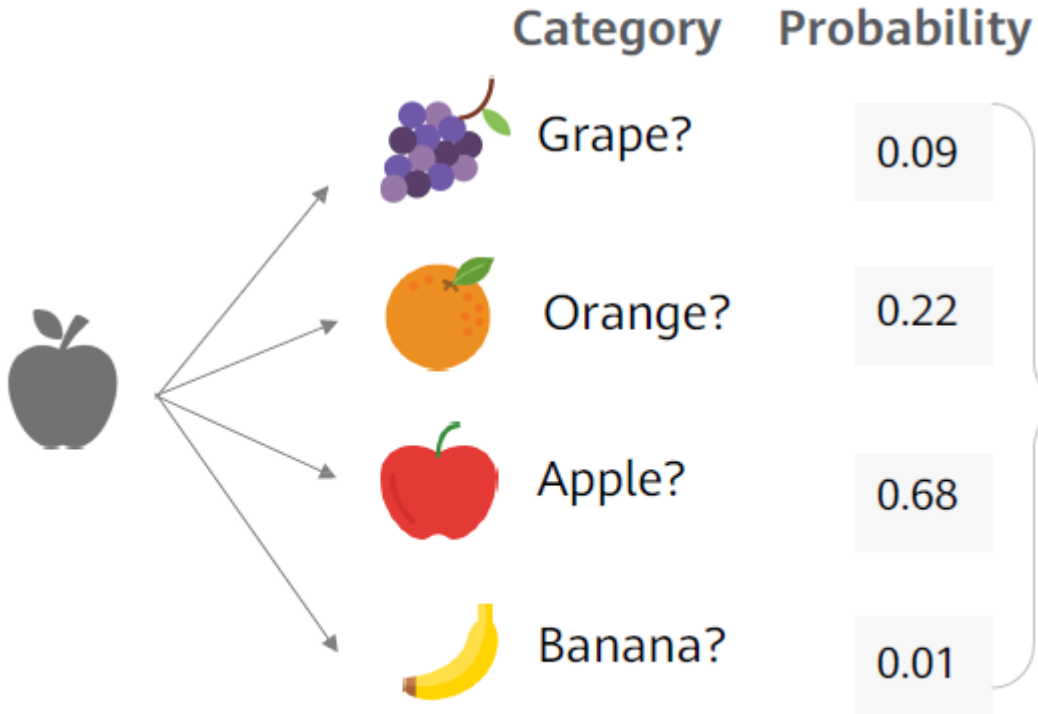
Logistic Regression Extension - Softmax Function (2)

- Softmax regression is a generalization of logistic regression that we can use for K-class classification.
- The Softmax function is used to map a K-dimensional vector of arbitrary real values to another K-dimensional vector of real values, where each vector element is in the interval (0, 1).
- The regression probability function of Softmax is as follows:

$$p(y=k \mid x; w)=\frac{e^{w_k^T x}}{\sum_{l=1}^{K} e^{w_l^T x}}, k=1,2\mathrm{L}\ \ ,K$$

Logistic Regression Extension - Softmax Function (3)

- Softmax assigns a probability to each class in a multi-class problem. These probabilities must add up to 1.
  - Softmax may produce a form belonging to a particular class. Example:



- **Sum of all probabilities:**
  - 0.09 + 0.22 + 0.68 + 0.01 =1
- Most probably, this picture is an apple.

Multilayer perceptron (MLP)

- Networks without hidden units are very limited in the input-output mappings they can learn to model.
  - More layers of linear units do not help. It is still linear.
  - Fixed output non-linearities are not enough.

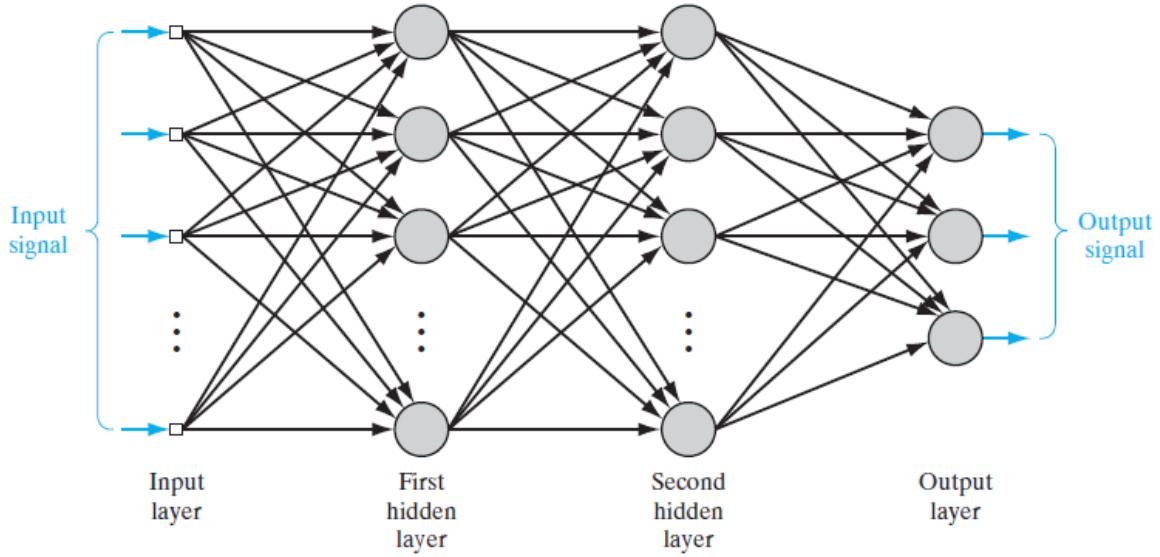
- The perceptron convergence procedure works by ensuring that every time the weights change, they get closer to every "generously feasible" set of weights.
  - This type of guarantee cannot be extended to more complex networks in which the average of two good solutions may be a bad solution.

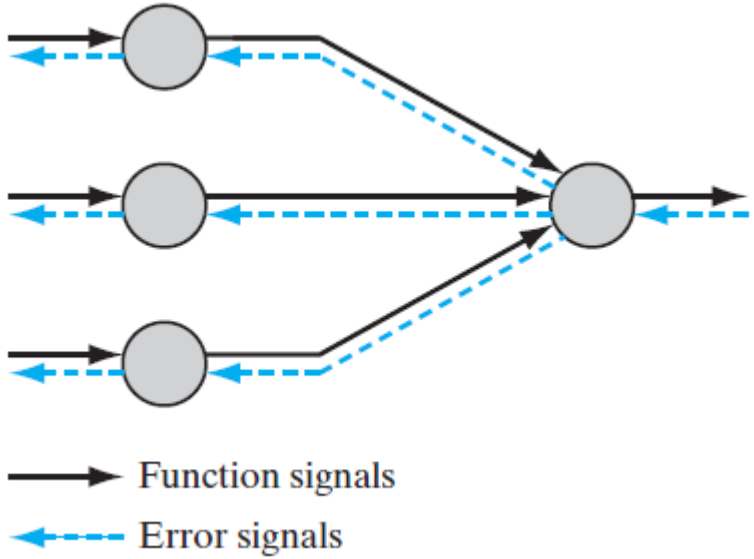
- So "multilayer" neural networks do not use the perceptron learning procedure.
  - They should never have been called multi-layer perceptrons.

We want a method that can be generalised to multi-layer, non-linear neural networks.

Multilayer perceptrons (MLP)

- Depending on the signal flow direction, there two kind of signals:
- **Function signals** – an input signal (stimulus) that comes in at the input end of the network, propagates **forward** (neuron by neuron) through the network, and **emerges** at the output end of the network as an output signal.
- **Error signal** – originates at an output neuron of the network and propagates **backward** (layer by layer) through the network.
- Function signals are used to **make predictions** whilst error signals, to **adapt the weights** (as function of backpropagated error)





Linear Regression (2)

- The model function of linear regression is as follows, where w indicates the weight parameter, b indicates the bias, and x indicates the sample attribute.

$$h_w(x) = w^T x + b$$

- The relationship between the value predicted by the model and actual value is as follows, where y indicates the actual value, and ε indicates the error.

$$y = w^T x + b + \varepsilon$$

- The error ε is influenced by many factors independently. According to the central limit theorem, the error ε follows normal distribution. According to the normal distribution function and maximum likelihood estimation, the loss function of linear regression is as follows:

$$J(w) = \frac{1}{2m} \sum \left(h_w(x) - y \right)^2$$

- To make the predicted value close to the actual value, we need to minimize the loss value. We can use the gradient descent method to calculate the weight parameter w when the loss function reaches the minimum, and then complete model building.

Logistic Regression (2)

- Both the logistic regression model and linear regression model are generalized linear models. Logistic regression introduces nonlinear factors (the sigmoid function) based on linear regression and sets thresholds, so it can deal with binary classification problems.
- According to the model function of logistic regression, the loss function of logistic regression can be estimated as follows by using the maximum likelihood estimation:

$$J(w) = -\frac{1}{m}\sum\left(y \ln h_w(x) + (1-y)\ln(1-h_w(x))\right)$$

- where w indicates the weight parameter, m indicates the number of samples, x indicates the sample, and y indicates the real value. The values of all the weight parameters w can also be obtained through the gradient descent algorithm.

The back-propagation algorithm

- **Data pre-processing** – It is recommended to **standardise** the data (mean=0, sd=1).

1. **Initialisation** – Unless there is prior information available, pick synaptic weights from uniform distribution whose mean is zero and variance smaller than 1 (**assuming** standardised data).
2. **Presentation** of training examples – Present the network an **epoch** of training examples. For each example in the sample perform the sequence of forward and backward computations:
3. **Forward computation** – Compute network outputs $\boldsymbol{y}(n)$ and errors $\mathrm{J}(n)$.
4. **Backward computation** – Compute the local gradients and then, adjust the weights as follows:

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \eta \frac{\partial J(w)}{\partial \mathbf{w}(n)},$$ where $\eta > 0$ is the *learning rate*

5. **Iteration** – Iterate the forward and backward computations under points 3 and 4 by presenting new epochs of training examples to the network until the **chosen stopping criterion** is met.

Gradient descent

- If we define the loss (or error) as a function of the weights, $\mathrm{J}(\mathbf{w})$, it makes sense to let the weights update proportionally to loss change:

$$\Delta \mathbf{w}^{(n)} \propto \frac{\partial J}{\partial \mathbf{w}(n)}$$

where $\Delta \mathbf{w}^{(n)} = \mathbf{w}(n+1) - \mathbf{w}(n)$, and n labels the iteration step, so:

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \eta \frac{\partial J}{\partial \mathbf{w}(n)}, \qquad \text{where } \eta > 0 \text{ is the } \textit{learning rate}.$$

- After each such update, the gradient is re-evaluated for the new weight vector and the process repeated.
- Note that the loss function is defined with respect to a training set, and so each step requires that the entire training set be processed in order to evaluate the gradient.
- The gradient $\frac{\partial J}{\partial \mathbf{w}(n)}$ is estimated using the back-propagation algorithm.

An analogy for understanding gradient descent [*]

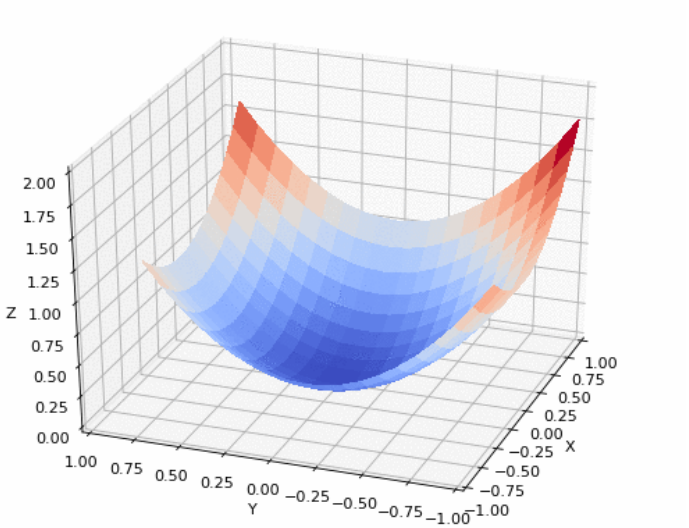
- A person is stuck in the mountains and is trying to get down.

- There is heavy fog and the path down the mountain is not visible, so he must use local information to get down.
- He can use the method of **gradient descent**, which involves looking at the steepness of the hill at his current position, then proceeding in the direction with the steepest descent. Using this method, he would eventually find his way down the mountain.
- Assume that the steepness of the hill is not immediately obvious with simple observation, but rather it requires a sophisticated instrument to measure, which the person happens to have at the moment.
- It takes quite some time to measure the steepness of the hill with the instrument, thus he should minimise its use. The difficulty then is choosing the frequency at which he should measure the steepness of the hill so not to go off track.

[*] https://en.wikipedia.org/wiki/Backpropagation

An analogy for understanding gradient descent [*]

- A person is stuck in the mountains and is trying to get down (i.e. trying to find the minima).



- In this analogy:
  - the person: the backpropagation algorithm, and the path taken down the mountain represents the sequence of parameter settings that the algorithm will explore.
  - the steepness of the hill: the slope of the error surface at that point.
  - the instrument used to measure steepness: differentiation (the slope of the error surface can be calculated by taking the derivative of the squared error function at that point).
  - the direction he chooses to travel in: gradient of the error surface at that point.
  - the amount of time he travels before taking another measurement: the learning rate of the algorithm.

[*] https://en.wikipedia.org/wiki/Backpropagation

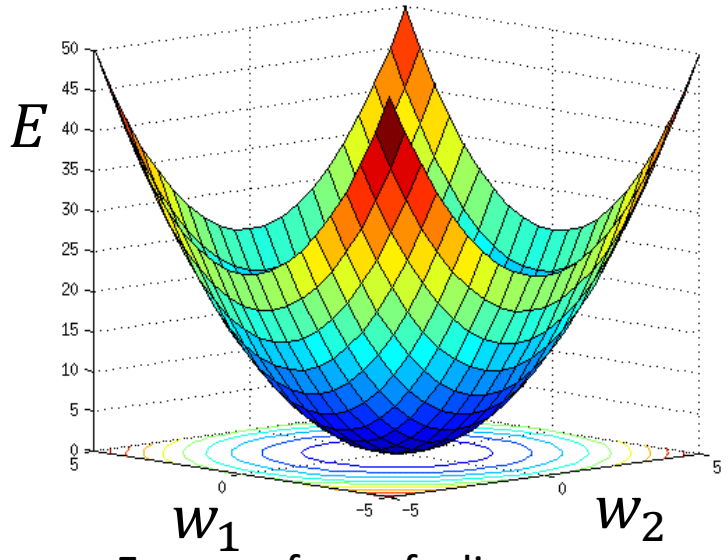
The error surface for a linear neuron

The error surface means a surface that lies in a space where the horizontal axes correspond to the weights of the neural net, and the vertical axis corresponds to the error it makes.

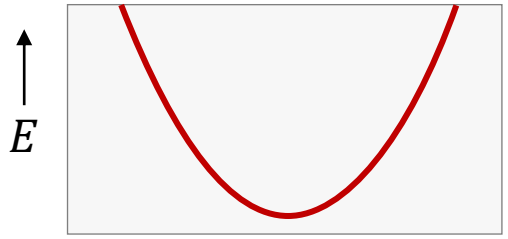
- For a linear neuron with a squared error, that surface always forms a quadratic bowl.
- Vertical cross sections are parabolas.
- Horizontal cross sections are ellipses.

For multilayer non linear nets the error surface is much more complicated.

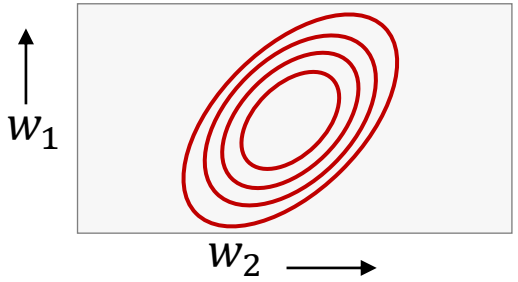
- As long as the weights are not too big, the error surface will still be smooth, but it may have many local minima.



Error surface of a linear neuron with two input weights



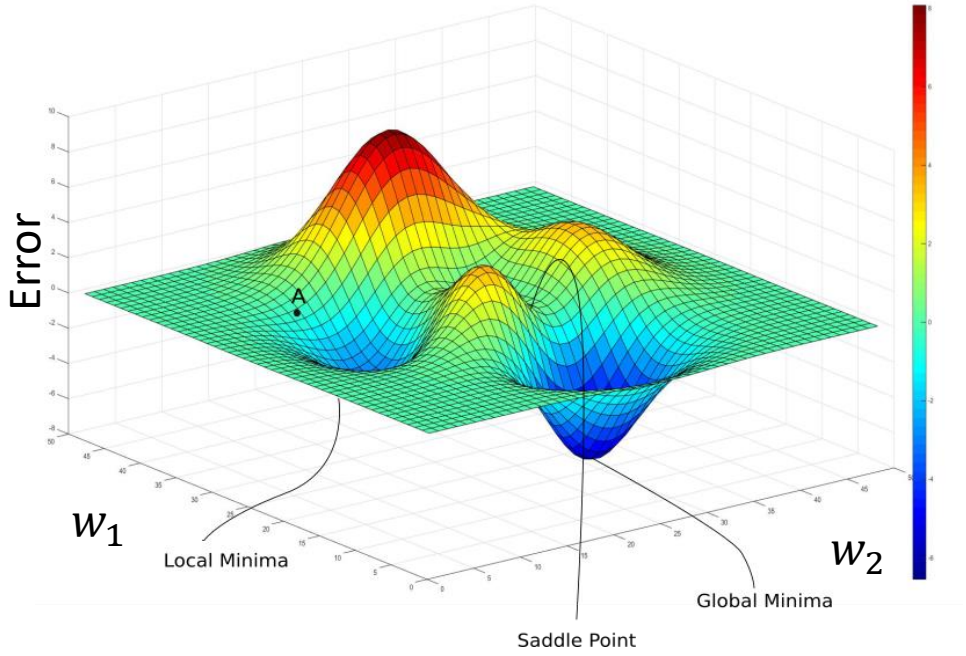
Vertical cross section



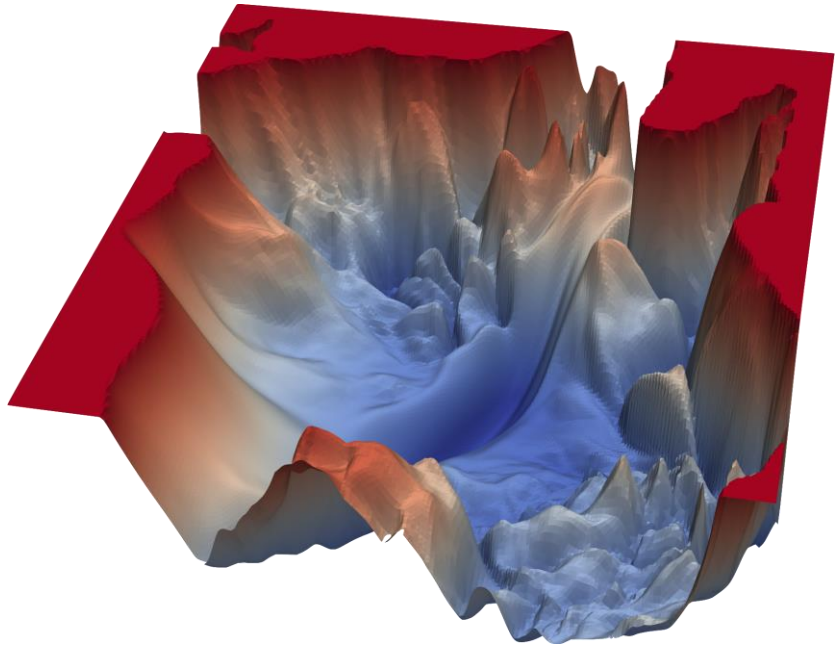
Horizontal cross sections

Error surface

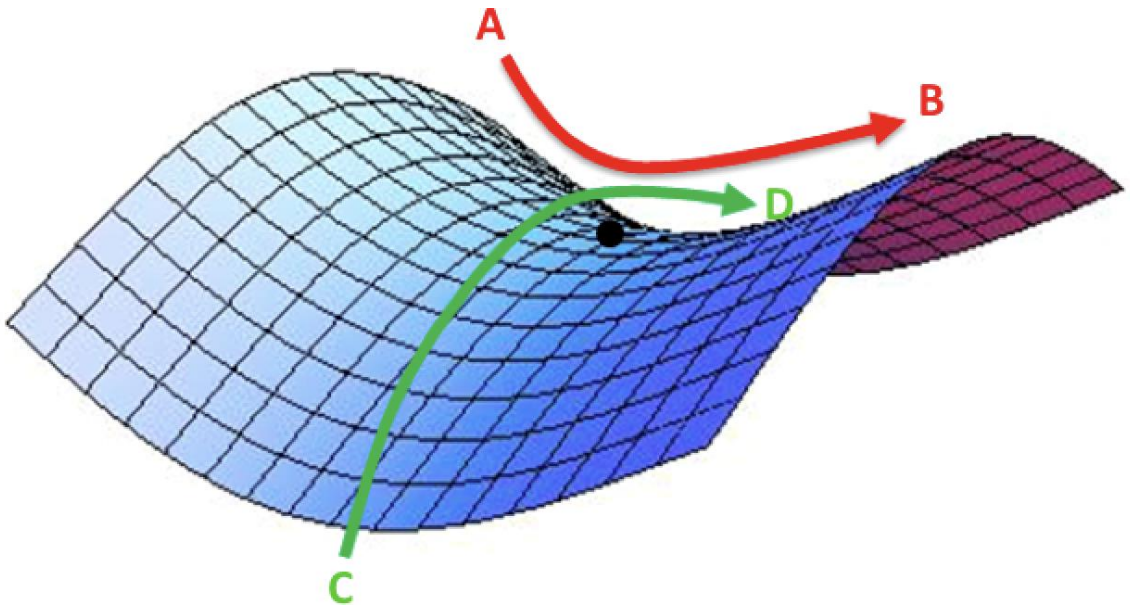
- An error surface is produced a function of the “error” (e.g. RSS) vs weights.
- In general, it is not possible to plot one unless we had one neuron with 1 to 3 inputs.
- Consequently, we don’t know where the minima are in advance.



- Actual error surfaces are quite bumpy with many local minima and saddle points
- Gradient descent can’t find the global minima or tell whether it is or not.



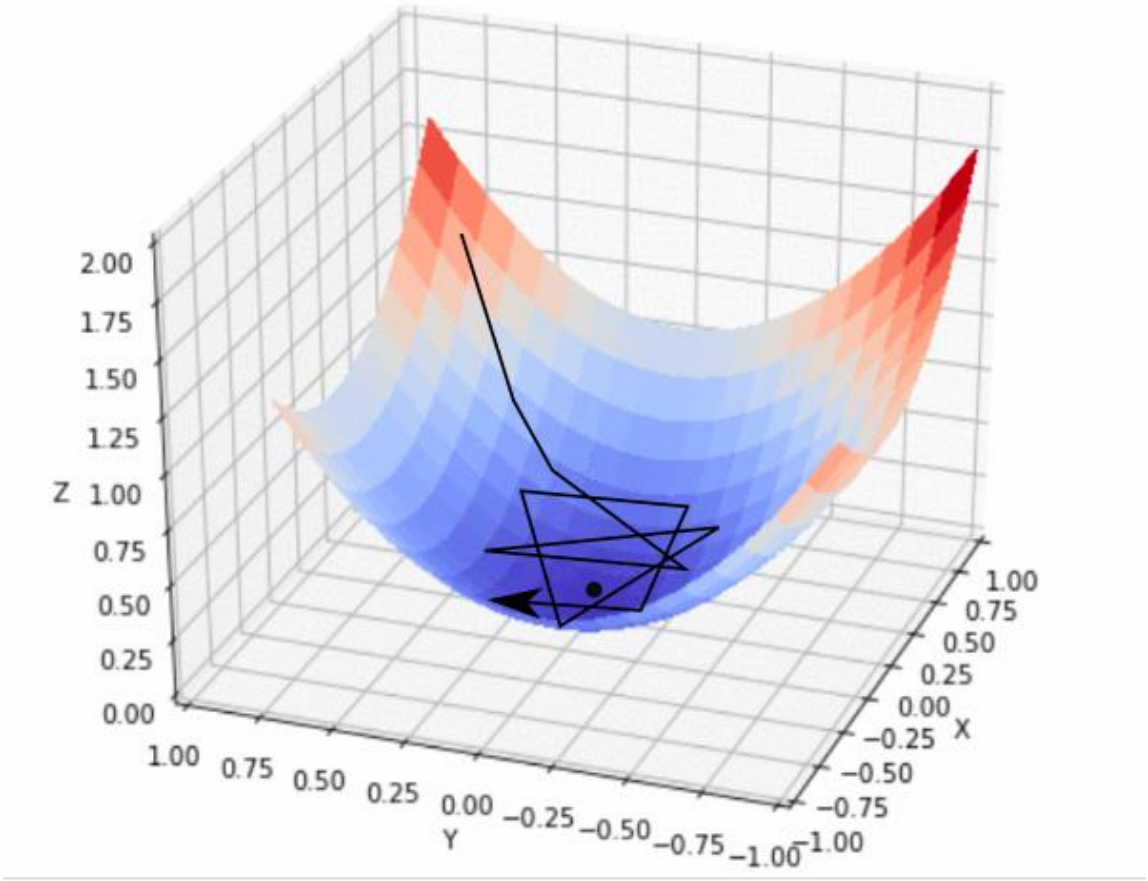
Saddle point



- A saddle point is a minima in one direction and a maxima in another direction.
- It is problematic because makes gradient descent to keep oscillating along the direction of the maxima while giving the illusion that it has converged to a minima.
- Again, as with the local minima, we don’t know where they are.

Learning rate

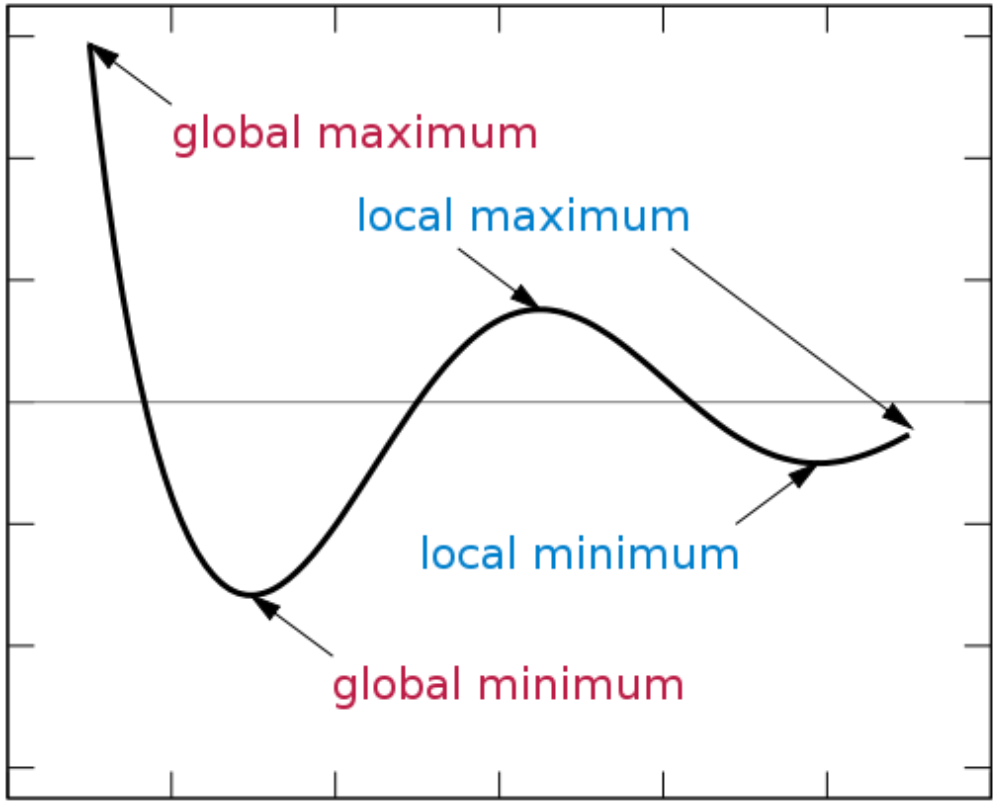
- The learning rate **controls** the speed of the learning process.
- Too high: in some cases network could learn fast but better solutions could be missed. Also, the opposite effect could happen: it could keep bouncing along the ridges of the "valley" without ever reaching the minima.
- Too low: the training might turn out to be too long to be feasible at all. Even if that is not the case, very slow learning rates make the algorithm more prone to get stuck in a minima.



Too fast.

Limitation of gradient descent with backpropagation

- Gradient descent with backpropagation is not guaranteed to find the global minimum of the error function, but only a local minimum.
- Also, it has trouble crossing plateaux in the error function landscape.
- This issue, caused by the non-convexity of error functions in neural networks, was long thought to be a major drawback.
- But in a 2015 review article[*], the authors argue that in many practical problems, it is not.



[*] LeCun, et. al, "Deep learning" Nature, 521(7553):436-444, 2015.

Implementing MLPs with Scikit-Learn

```
from sklearn.neural_network import MLPClassifier

mlp_mdl = MLPClassifier(hidden_layer_sizes=(5,),random_state=42,
max_iter=300)

mlp_mdl.fit(X_train, y_train)


mlp_mdl.predict(X_test)
```

Summary

- Modelling with Artificial Neural Networks (ANNs).
- Perceptron as a linear binary classifier.
- Multi-layer perceptron as a non-linear algorithm.
- MLP output is decided by the problem:
  - Regression: 1 (likely linear) output neuron
  - Classification: # output neurons = # classes (or # classes – 1), activation functions: sigmoids
  - Hidden layers: (theory) 1 hidden layer is sufficient. (practice) complex tasks could benefit from using more hidden layers. Number of neurons: several options should be tested. Same for the selection of the activation functions, but ‘ReLU’s should be sufficient.
- Learning is an iterative process.